

“HỌC ĐỂ LÀM VIỆC”

C++ for Base

Contents

1

Array in C++

2

Strings

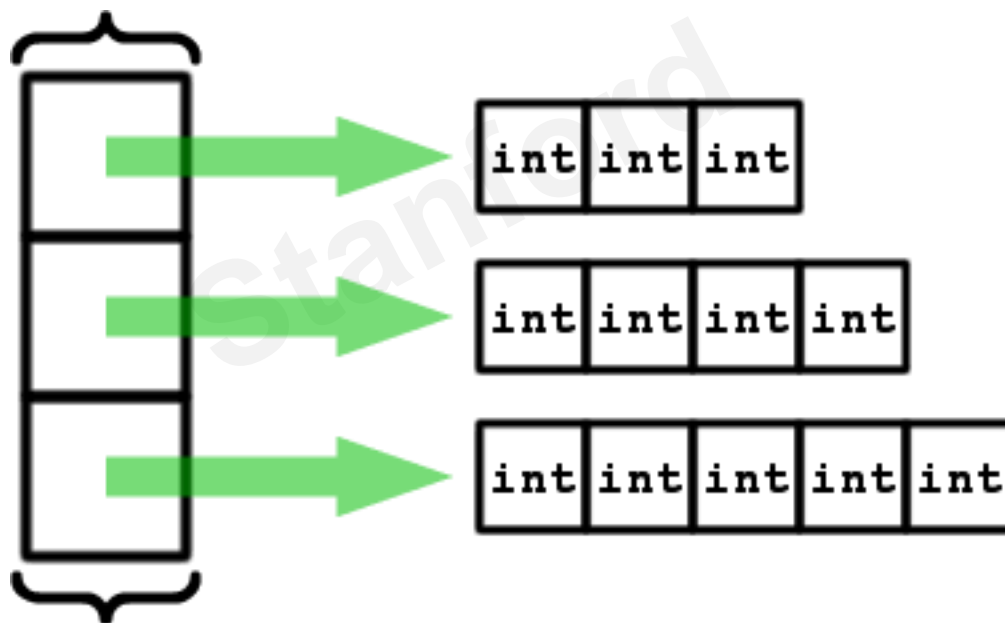
3

Exercises

Array in C++

❖ Array in C++

- **Mảng (array):** là tập hợp các phần tử có cùng kiểu dữ liệu



❖ Array in C++

Ví dụ: Mảng 2 chiều gồm 3 hàng và 3 cột

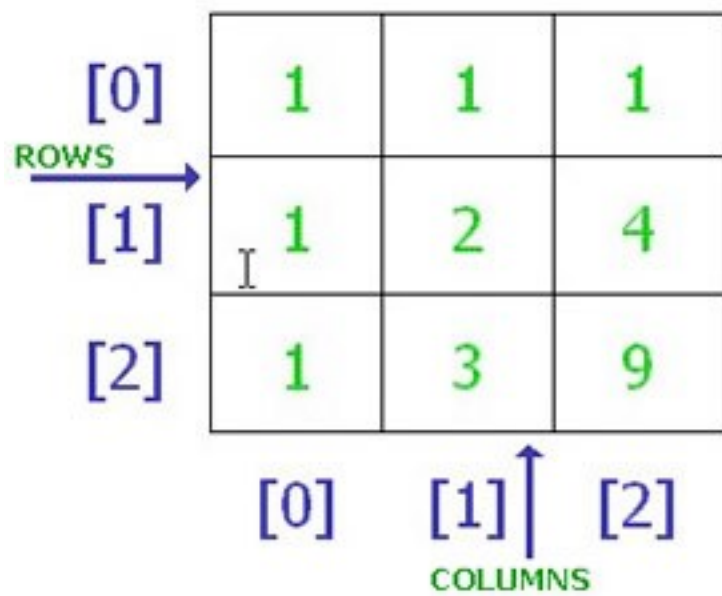
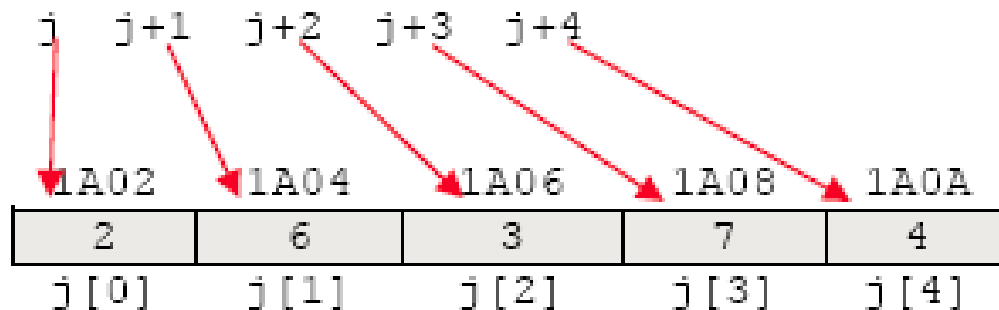


Diagram illustrating a 2D array structure with 3 rows and 3 columns. The rows are indexed [0], [1], [2] and the columns are indexed [0], [1], [2]. The values in the array are:

[0]	1	1	1
[1]	1	2	4
[2]	1	3	9
	[0]	[1]	[2]

Labels: ROWS (pointing to rows), COLUMNS (pointing to columns).



❖ Array in C++

■ Mảng 1 chiều:

- Mảng chứa nhiều phần tử có cùng kiểu dữ liệu.
- Cú pháp khai báo mảng:
- **<Kiểu dữ liệu> <Tên mảng>[Số phần tử của mảng];**

Ví dụ:

```
int a[2]; //Khai báo mảng có 2 phần tử
```

```
int a[2] = {6, 8} // Gán giá trị có các phần tử
```

```
a[0] = 6; a[1] = 8;
```

❖ Array in C++

- **Mảng 1 chiều:**

- **Ví dụ 1:** Khai báo Mảng gồm n phần tử có kiểu int do người sử dụng nhập vào. Sau đó yêu cầu người sử dụng nhập giá trị cho n phần tử đó → in chúng ra ngoài màn hình

❖ Array in C++

✓ Mảng nhiều chiều:

➤ Mảng 2 chiều:

■ Cú pháp:

- <Kiểu dữ liệu><Tên Mảng>[m][n];
- m: Số dòng được tính theo hàng ngang của Mảng.
- n: Số cột được tính theo hàng dọc của Mảng.
- Ví dụ: Khai báo mảng 2 chiều:

```
int arr[2][3] = {{3, 4, 5}, {6, 8, 9}};
```


❖ Array in C++

✓ Mảng nhiều chiều:

➤ Mảng 2 chiều:

- **Ví dụ 1:** Khai báo Mảng 2 chiều, đưa giá trị vào mảng rồi in chúng ra ngoài màn hình.

```
//Khai báo biến  
float a[3][2];
```

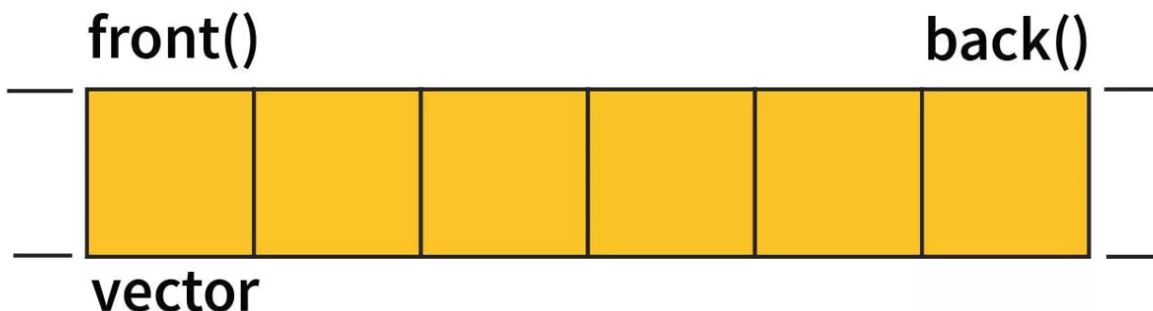
```
//Gán giá trị cho hàng 1  
a[0][0] = 2.5; a[0][1] = 3.5;  
//Hàng thứ 2  
a[1][0] = 5.2; a[1][1] = 6.8;  
//Hàng thứ 3  
a[2][0] = 8.6; a[2][1] = 9.5;
```

```
//In thông tin ra màn hình  
for(int i=0; i<3; i++)  
{  
    for(int j=0; j<2; j++)  
    {  
        cout <<a[i][j]<<"\t";  
    }  
  
    cout <<endl;  
}
```

❖ Array in C++

✓ Vector:

- **Vector:** Là một kiểu mảng động trong lập trình c++. Số lượng phần tử có thể thay đổi tùy theo mục đích sử dụng của người dùng
- Khai báo thư viện: `#include<vector>` để sử dụng vector



❖ Danh sách trong C++

✓ Vector:

- **Vector:** Là một kiểu mảng động trong lập trình c++. Số lượng phần tử có thể thay đổi tùy theo mục đích sử dụng của người dùng

❑ Ví dụ: Khai báo 1 danh sách số nguyên như sau

```
//Khai báo khởi tạo 1 danh sách  
vector<int> vec1 = { 1, 2, 5, 6, 8 };  
  
vector<int> vec2{ 2, 3, 6 };  
  
//Sử dụng hàm  
vector<int> vec3(3, 5);
```

❖ Danh sách trong C++

✓ Vector:

- **Vector:** Là một kiểu mảng động trong lập trình c++. Số lượng phần tử có thể thay đổi tùy theo mục đích sử dụng của người dùng

❑ Ví dụ: Khai báo 1 danh sách số nguyên như sau

```
cout << "Danh sach cac phan tu trong vec 1: " << endl;  
  
for (const int& i : vec1)  
{  
    cout << i << endl;  
}
```

❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

- **push_back(value)**: Để thêm một giá trị vào 1 vector
- **size()**: Trả về số lượng các phần tử trong vector
- **empty()**: Trả về true nếu danh sách rỗng
- **front()**: Hàm trả về giá trị đầu tiên trong danh sách
- **back()**: Hàm trả về giá trị cuối cùng trong danh sách
- **at(pos)**: Hàm trả về giá trị của phần tử theo vị trí trong danh sách
- **data()**: Hàm trả về dữ liệu danh sách về dạng con trỏ

❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

- **begin()**: Hàm trả về vị trí đầu tiên của danh sách
- **end()**: Hàm trả về vị trí cuối của danh sách
- **rbegin()**: Hàm trả về hằng số con trỏ vị trí cuối trong danh sách
- **rend()**: Hàm trả về hằng số con trỏ vị trí đầu trong danh sách
- **cbegin()**: Hàm trả về hằng số con trỏ vị trí đầu trong danh sách
- **cend()**: Hàm trả về hằng số con trỏ vị trí đầu trong danh sách

❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

- **assign(n, m):** Tạo danh sách gồm n phần tử với giá trị m trong vector
- **pop_back():** Xóa phần tử cuối trong danh sách
- **insert():** Chèn 1 giá trị vào vị trí con trỏ nào đó trong danh sách
- **erase():** Xóa phần tử trong danh sách
- **swap():** Hàm thực hiện hoán đổi 2 danh sách vector cùng kiểu
- **clear():** Xóa các phần tử trong danh sách
- **emplace():** Thêm phần tử vào vị trí nào đó trong danh sách
- **emplace_back():** Thêm phần tử vào cuối danh sách

❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

▪ Ví dụ:

```
//Khai báo 1 vector
vector<int> vec;

//Thêm giá trị
vec.push_back(3);
vec.push_back(5);
vec.push_back(7);
cout << "Số lượng phần tử: " << vec.size() << endl;
cout << "Phần tử đầu: " << vec.front() << endl;
cout << "Phần tử cuối: " << vec.back() << endl;
cout << "Phần tử ở vị trí thứ 2 là: " << vec.at(1) << endl;
```


❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

▪ Ví dụ:

```
//Di chuyển trở về vị trí đầu rồi thêm số 8  
vec.insert(vec.begin(), 8);
```

```
//Tạo danh sách 4 phần tử với các giá trị 5  
vec.assign(4, 5);
```

```
//Xóa phần tử cuối  
vec.pop_back();
```

❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

▪ Ví dụ:

```
//Chèn vào vị trí con trỏ đầu của vector giá trị 6  
vec.emplace(vec.begin(), 6);
```

```
//Chèn vào phần tử cuối danh sách  
vec.emplace_back(10);
```

```
//Xóa phần tử ở vị trí đầu  
vec.erase(vec.begin());
```

❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

▪ Ví dụ:

```
cout << "Cac phan tu: " << endl;  
  
for (auto i = vec.begin(); i != vec.end(); ++i)  
{  
    cout << *i << endl;  
}  
  
//Xóa tất cả các phần tử  
vec.clear();
```

```
So luong phan tu: 3  
Phan tu dau: 3  
Phan tu cuoi: 7  
Phan tu o vi tri thu 2 la: 5  
Cac phan tu:  
5  
5  
5  
10
```

Output

❖ Danh sách trong C++

✓ Vector:

➤ Các hàm sử dụng trong Vector:

▪ Ví dụ:

```
vector<int> vec4 = { 1, 2 };
```

```
vector<int> vec5 = { 3, 4 };
```

```
//Hoán đổi  
vec4.swap(vec5);
```

```
Danh sach cac phan tu trong vec 4:  
3  
4  
Danh sach cac phan tu trong vec 5:  
1  
2
```

Output

String

❖ String in C++

- ✓ Để làm việc với chuỗi trong C++ người sử dụng cần khai báo: **#include <string.h>**
- ✓ **strcpy(s, t)**: Gán nội dung của xâu t cho xâu s (thay cho phép gán = không được dùng)

Ví dụ:

```
char s[10], t[10] ;  
//t = "Stanford" ;    // không được dùng  
//s = t ;              // không được dùng  
strcpy(t, "Stanford") ;    //được, gán "Stanford" cho t  
strcpy(s, t) ;            // được, sao chép t sang s  
cout << s << " to " << t ;    // in ra: Stanford to Stanford
```

❖ String in C++

- ✓ **strncpy(s, t, n):** Sao chép n kí tự của t vào s. Hàm này chỉ làm nhiệm vụ sao chép, không tự động gắn kí tự kết thúc chuỗi cho s. Do vậy người dùng phải thêm câu lệnh đặt kí tự '\0' vào cuối chuỗi s sau khi sao chép xong.

Ví dụ:

```
char s[10], t[10] = "Stanford";  
strncpy(s, t, 5) ; // copy 5 kí tự "Stanford" vào s  
s[5] = '\0' ;      // đặt dấu kết thúc chuỗi
```

❖ String in C++

- ✓ **strcat(s, t):** Nối một bản sao của t vào sau s (thay cho phép +)

Ví dụ:

```
char s[100] , t[100] = "Steve" ;  
strncpy(s, t, 3); s[3] = '\0';           // s = "Ste"  
strcat(s, "p");                          // s = "Step"  
cout << t << " goes " << s << " by " << s // Steve goes Step by Step
```


❖ String in C++

- ✓ **strcmp(s, t):** Hàm so sánh 2 chuỗi s và t (thay cho các phép toán so sánh). Giá trị trả lại là hiệu 2 kí tự khác nhau đầu tiên của s và t. Từ đó, nếu $s1 < s2$ thì hàm trả lại giá trị âm, bằng 0 nếu $s1 == s2$, và dương nếu $s1 > s2$.

Ví dụ:

```
if (strcmp(s,t)) cout << "s khác t"; else cout << "s bằng t" ;
```

❖ String in C++

- ✓ **strncmp(s, t, n):** Giống hàm strcmp(s, t) nhưng chỉ so sánh tối đa n kí tự đầu tiên của hai chuỗi.

Ví dụ:

```
char s[] = "Hà Nội" , t[] = "Hà nội" ;
```

```
cout << strcmp(s,t) ;    // -32 (vì 'N' = 78, 'n' = 110)
```

```
cout << strncmp(s, t, 3) ; // 0 (vì 3 kí tự đầu của s và t là  
như nhau)
```

❖ String in C++

- ✓ **strcmpi(s, t)**: Như strcmp(s, t) nhưng không phân biệt chữ hoa, thường.

Ví dụ:

```
char s[] = "Hà Nội" , t[] = "hà nội" ;
```

```
cout << strcmpi(s, t) ;    // 0 (vì s = t)
```

❖ String in C++

- ✓ **strupr(s)**: Hàm đổi chuỗi s thành in hoa và cũng trả lại chuỗi in hoa đó.

Ví dụ:

```
char s[10] = "Ha noi" ;
```

```
cout << strupr(s) ;    // HA NOI
```

```
cout << s ;           // HA NOI (s cũng thành in hoa)
```

❖ String in C++

- ✓ **strlwr(s)**: Hàm đổi chuỗi s thành in thường, kết quả trả lại là chuỗi s.

Ví dụ:

```
char s[10] = "Ha Noi" ;
```

```
cout << strlwr(s) ; // ha noi
```

```
cout << s ; // ha noi (s cũng thành in thường)
```

❖ String in C++

- ✓ **strlen(s)**: Hàm trả giá trị là độ dài của xâu s.

Ví dụ:

```
char s[10] = "Ha Noi" ;
```

```
cout << strlen(s) ; // 5
```

Exercises

“HỌC ĐỂ LÀM VIỆC”

Thank You !